

IPC THREAD SYNCHRONIZATION

1. Synchronisation Using Mutexes

1. Write a C program where multiple threads increment a shared counter using a mutex to prevent race conditions.

PROGRAM:

```
/*
Program to demonstrate mutex synchronization (shared counter)
Author   : MUTHUGANESH S
Date     : 14/03/2026
Filename : cou.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>

// Shared counter variable and mutex lock
int counter = 0;
pthread_mutex_t lock;

// Function to increment the counter
void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    printf("Final Counter Value: %d\n", counter);
}
```

```

    return 0;
}

```

OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc cou.c -o cou
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./cou
Final Counter Value: 200000

```

2. Implement a producer-consumer problem using a mutex to synchronize access to a shared buffer.

PROGRAM:

```

/*
Program to demonstrate producer-consumer using mutex
Author   : MUTHUGANESH S
Date     : 14/03/2026
Filename : prod.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Buffer size and shared variables
#define SIZE 5

// Shared buffer and count of items in the buffer
int buffer[SIZE];
int count = 0;

// Mutex lock for synchronizing access to the buffer
pthread_mutex_t lock;

// Producer thread function
void* producer(void* arg) {
    for (int i = 1; i <= 10; i++) {
        pthread_mutex_lock(&lock);

        if (count < SIZE) {
            buffer[count++] = i;
            printf("Produced: %d\n", i);
        }
    }
}

```

```

        pthread_mutex_unlock(&lock);
        sleep(1);
    }
    return NULL;
}

// Consumer thread function
void* consumer(void* arg) {
    for (int i = 1; i <= 10; i++) {
        pthread_mutex_lock(&lock);

        if (count > 0) {
            int item = buffer[--count];
            printf("Consumed: %d\n", item);
        }

        pthread_mutex_unlock(&lock);
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc alt.c -o
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./alt
Odd Thread: 1
Even Thread: 2
Odd Thread: 3
Even Thread: 4
Odd Thread: 5
Even Thread: 6
Odd Thread: 7
Even Thread: 8
Odd Thread: 9
Even Thread: 10
Odd Thread: 11
Even Thread: 12
Odd Thread: 13
Even Thread: 14
Odd Thread: 15
Even Thread: 16
Odd Thread: 17
Even Thread: 18
Odd Thread: 19
Even Thread: 20
Odd Thread: 21
mg@MG:/mnt/m/training/LINUX/IPC THREAD$
```

3. Create a program where two threads alternately print numbers from 1 to 20, using a mutex to control access to the print operation.

PROGRAM:

```
/*
Program to print numbers alternately using mutex
Author   : MUTHUGANESH S
Date     : 14/03/2026
Filename : alt.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>

// Shared variable and mutex
int num = 1;
pthread_mutex_t lock;

// Function to print odd numbers
void* printOdd(void* arg) {
    while (num <= 20) {
        pthread_mutex_lock(&lock);
```

```

        if (num % 2 != 0) {
            printf("Odd Thread: %d\n", num);
            num++;
        }

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

// Function to print even numbers
void* printEven(void* arg) {
    while (num <= 20) {
        pthread_mutex_lock(&lock);

        if (num % 2 == 0) {
            printf("Even Thread: %d\n", num);
            num++;
        }

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, printOdd, NULL);
    pthread_create(&t2, NULL, printEven, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc prod.c -o prod
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./prod
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
mg@MG:/mnt/m/training/LINUX/IPC THREAD$
```

2. Synchronisation Using Conditional Variables

1. Write a C program that uses a condition variable to signal a consumer thread when a producer thread adds data to a shared buffer.

PROGRAM:

```
/*
Program to demonstrate producer-consumer using condition
variable
Author   : MUTHUGANESH S
Date     : 12/03/2026
Filename : pc.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Define constants for buffer size
#define SIZE 5

// Buffer and count
```

```

int buffer[SIZE];
int count = 0;

// Mutex and condition variable for synchronization
pthread_mutex_t mutex;
pthread_cond_t cond;

// Producer thread function
void* producer(void* arg) {
    for (int i = 1; i <= 10; i++) {
        pthread_mutex_lock(&mutex);

        while (count == SIZE)
            pthread_cond_wait(&cond, &mutex);

        buffer[count++] = i;
        printf("Produced: %d\n", i);

        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);

        sleep(1);
    }
    return NULL;
}

// Consumer thread function
void* consumer(void* arg) {
    for (int i = 1; i <= 10; i++) {
        pthread_mutex_lock(&mutex);

        while (count == 0)
            pthread_cond_wait(&cond, &mutex);

        int item = buffer[--count];
        printf("Consumed: %d\n", item);

        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);

        sleep(2);
    }
    return NULL;
}

// Main function
int main() {
    pthread_t p, c;

```

```
pthread_mutex_init(&mutex, NULL);
pthread_cond_init(&cond, NULL);

pthread_create(&p, NULL, producer, NULL);
pthread_create(&c, NULL, consumer, NULL);

pthread_join(p, NULL);
pthread_join(c, NULL);

pthread_mutex_destroy(&mutex);
pthread_cond_destroy(&cond);

return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc pc.c -o pc
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./pc
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 4
Produced: 5
Produced: 6
Consumed: 6
Produced: 7
Produced: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
Consumed: 8
Consumed: 7
Consumed: 5
Consumed: 3
mg@MG:/mnt/m/training/LINUX/IPC THREAD$
```


2. Implement a thread pool where worker threads wait on a condition variable for new tasks.

PROGRAM:

```
/*
Program to demonstrate thread pool using condition variable
Author   : MUTHUGANESH S
Date     : 12/03/2026
Filename : tp.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Define constants for the number of threads and tasks
#define THREADS 3
#define TASKS 5

// Task queue and count
int task_queue[TASKS];
int task_count = 0;

// Mutex and condition variable for synchronization
pthread_mutex_t mutex;
pthread_cond_t cond;

// Worker thread function
void* worker(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        while (task_count == 0)
            pthread_cond_wait(&cond, &mutex);

        int task = task_queue[--task_count];
        printf("Thread %ld processing task %d\n", (long)arg,
task);

        pthread_mutex_unlock(&mutex);

        sleep(1);
    }
    return NULL;
}
```

```

int main() {
    pthread_t workers[THREADS];

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);

    for (long i = 0; i < THREADS; i++)
        pthread_create(&workers[i], NULL, worker, (void*)i);

    for (int i = 1; i <= TASKS; i++) {
        pthread_mutex_lock(&mutex);

        task_queue[task_count++] = i;
        printf("Added task %d\n", i);

        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);

        sleep(1);
    }

    sleep(5);
    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc tp.c -o tp
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./tp
Added task 1
Thread 0 processing task 1
Added task 2
Thread 1 processing task 2
Added task 3
Thread 1 processing task 3
Added task 4
Thread 1 processing task 4
Added task 5
Thread 2 processing task 5
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ |

```

3. Create a program where one thread waits for a specific condition (e.g., counter reaches 10) before proceeding, using a condition variable for synchronization.

PROGRAM:

```
/*
Program to demonstrate waiting for a condition using condition
variable
Author   : MUTHUGANESH S
Date     : 12/03/2026
Filename : cw.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Shared counter variable
int counter = 0;

// Mutex and condition variable for synchronization
pthread_mutex_t mutex;
pthread_cond_t cond;

// Incrementer thread function
void* incrementer(void* arg) {
    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutex);

        counter++;
        printf("Counter: %d\n", counter);

        if (counter == 10)
            pthread_cond_signal(&cond);

        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

// Waiter thread function
void* waiter(void* arg) {
    pthread_mutex_lock(&mutex);

    while (counter < 10)
        pthread_cond_wait(&cond, &mutex);
}
```

```

        printf("Counter reached 10. Proceeding...\n");

        pthread_mutex_unlock(&mutex);
        return NULL;
    }

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);

    pthread_create(&t1, NULL, incrementer, NULL);
    pthread_create(&t2, NULL, waiter, NULL);

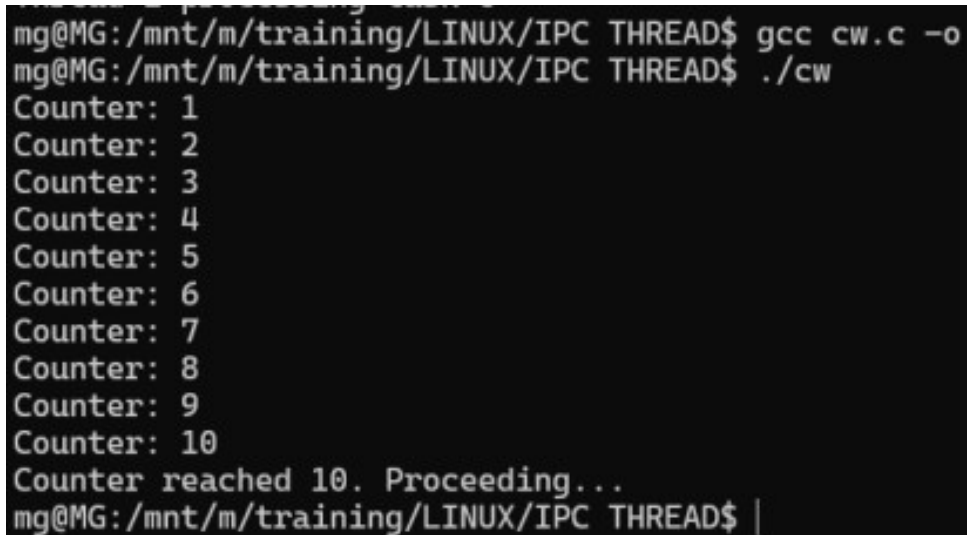
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);

    return 0;
}

```

OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc cw.c -o
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./cw
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
Counter: 6
Counter: 7
Counter: 8
Counter: 9
Counter: 10
Counter reached 10. Proceeding...
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ |

```

3. IPC - Unnamed Pipes

1. Write a C program where a parent process sends a string to its child process using an unnamed pipe.

PROGRAM:

```
/*
Program to demonstrate parent to child communication using
unnamed pipe
Author   : MUTHUGANESH S
Date    : 12/03/2026
Filename : ppc.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pid_t pid;
    char write_msg[] = "Hello from parent!";
    char read_msg[50];

    // Create a pipe
    pipe(fd);
    // Fork the process
    pid = fork();

    if (pid > 0) {
        close(fd[0]);
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);
    } else {
        close(fd[1]);
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);
    }

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc ppc.c -o ppc
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./ppc
Child received: Hello from parent!
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ |
```

2. Implement a program where two child processes communicate through an unnamed pipe created by their parent.

PROGRAM:

```
/*
Program to demonstrate communication between two child
processes using unnamed pipe
Author   : MUTHUGANESH S
Date     : 12/03/2026
Filename : tcp.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

// Main function
int main() {
    int fd[2];
    // Create two child processes
    pid_t c1, c2;
    char msg[] = "Message from Child 1";
    char buffer[100];

    // Create an unnamed pipe
    pipe(fd);

    // Fork the first child process
    c1 = fork();

    // Check if the first child process is created
    // successfully
    if (c1 == 0) {
        close(fd[0]);
        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);
    } else {
```

```

        c2 = fork();

        if (c2 == 0) {
            close(fd[1]);
            read(fd[0], buffer, sizeof(buffer));
            printf("Child 2 received: %s\n", buffer);
            close(fd[0]);
        } else {
            close(fd[0]);
            close(fd[1]);
            wait(NULL);
            wait(NULL);
        }
    }

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc tcp.c -o tcp
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./tcp
Child 2 received: Message from Child 1
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ |

```

3. Create a program that uses an unnamed pipe to transfer the contents of a file from one process to another.

PROGRAM:

```

/*
Program to transfer file contents using unnamed pipe
Author   : MUTHUGANESH S
Date     : 12/03/2026
Filename : fp.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;

```

```

char buffer[100];
int n;

pipe(fd);
pid = fork();

if (pid > 0) {
    int file = open("input.txt", O_RDONLY);

    close(fd[0]);

    while ((n = read(file, buffer, sizeof(buffer))) > 0) {
        write(fd[1], buffer, n);
    }

    close(file);
    close(fd[1]);
    wait(NULL);
} else {
    int file = open("output.txt", O_WRONLY | O_CREAT |
O_TRUNC, 0644);

    close(fd[1]);

    while ((n = read(fd[0], buffer, sizeof(buffer))) > 0)
{
        write(file, buffer, n);
    }

    close(file);
    close(fd[0]);
}

return 0;
}

```

OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc fp.c -o fp
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./fp
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

```


4. IPC - Named Pipes

1. Write a C program that creates a named pipe (FIFO) and uses it for communication between two unrelated processes.

PROGRAM:

```
/*
Program to demonstrate FIFO communication (Client)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : fifo_client.c
*/

//Header files
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char message[] = "Hello from client";

    //Open the FIFO for writing
    fd = open("myfifo", O_WRONLY);

    write(fd, message, sizeof(message));

    close(fd);

    return 0;
}
/*
Program to demonstrate FIFO communication (Server)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : fifo_server.c
*/

//Header files
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
```

```

char buffer[100];

//Create the FIFO (named pipe)
mkfifo("myfifo", 0666);

fd = open("myfifo", O_RDONLY);

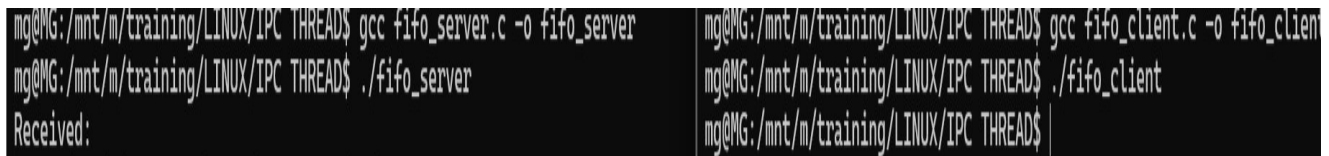
read(fd, buffer, sizeof(buffer));
printf("Received: %s\n", buffer);

close(fd);

return 0;
}

```

OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc fifo_server.c -o fifo_server
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./fifo_server
Received:
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc fifo_client.c -o fifo_client
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./fifo_client
Hello Server
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

```

2. Implement a client-server model using named pipes, where the client sends requests and the server responds.

PROGRAM:

```

/*
Program to demonstrate FIFO Client-Server (Client)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : cs_client.c
*/

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd1, fd2;
    char request[] = "Hello Server";
    char response[100];

    fd1 = open("req_fifo", O_WRONLY);
    write(fd1, request, sizeof(request));

    fd2 = open("res_fifo", O_RDONLY);
    read(fd2, response, sizeof(response));
}

```

```

        printf("Server Response: %s\n", response);

        close(fd1);
        close(fd2);

        return 0;
    }
    /*
    Program to demonstrate FIFO Client-Server (Server)
    Author   : MUTHUGANESH S
    Date      : 13/03/2026
    Filename  : cs_server.c
    */

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    int fd1, fd2;
    char request[100], response[100];

    mkfifo("req_fifo", 0666);
    mkfifo("res_fifo", 0666);

    fd1 = open("req_fifo", O_RDONLY);
    read(fd1, request, sizeof(request));

    printf("Client Request: %s\n", request);

    sprintf(response, "Processed: %s", request);

    fd2 = open("res_fifo", O_WRONLY);
    write(fd2, response, sizeof(response));

    close(fd1);
    close(fd2);

    return 0;
}

```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc cs_server.c -o cs_server
cs_server.c: In function 'main':
cs_server.c:26:35: warning: '%s' directive writing up to 99 bytes into a region of size 89 [-Wformat-overflow=]
   26 |     sprintf(response, "Processed: %s", request);
      |                               ^~
cs_server.c:26:5: note: 'sprintf' output between 12 and 111 bytes into a destination of size 100
   26 |     sprintf(response, "Processed: %s", request);
      |     ^~
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./cs_server
Client Request:

/home/mg/.hushlogin file.
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/IPC THREAD"
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc fifo_client.c -o fifo_client
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./fifo_client
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc cs_client.c -o cs_client
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./cs_client
Server Response:
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./cs_client
Server Response:
mg@MG:/mnt/m/training/LINUX/IPC THREAD$
```

3. Create a program that writes log messages to a named pipe, which are read and displayed by a separate process.

PROGRAM:

```
/*
Program to write logs to FIFO
Author    : MUTHUGANESH S
Date      : 13/03/2026
Filename  : writer.c
*/
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

int main() {
    int fd;
    int i = 1;
    char log[100];
    mkfifo("logfifo", 0666);
    fd = open("logfifo", O_WRONLY);

    while (1) {
        sprintf(log, "Log message %d\n", i++);
        write(fd, log, sizeof(log));
        sleep(1);
    }

    close(fd);
}
```

```

        return 0;
    }
    /*
    Program to read logs from FIFO
    Author   : MUTHUGANESH S
    Date      : 13/03/2026
    Filename  : reader.c
    */

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

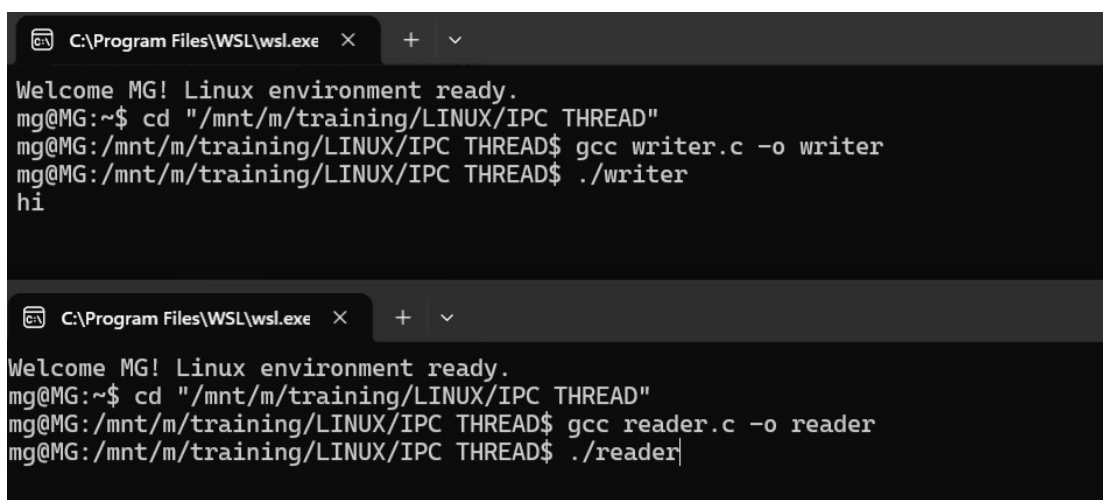
int main() {
    int fd;
    char buffer[100];

    fd = open("logfifo", O_RDONLY);

    while (1) {
        read(fd, buffer, sizeof(buffer));
        printf("Received Log: %s", buffer);
    }
    close(fd);
    return 0;
}

```

OUTPUT:



```

C:\Program Files\WSL\wsl.exe x + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/IPC THREAD"
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc writer.c -o writer
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./writer
hi

C:\Program Files\WSL\wsl.exe x + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/IPC THREAD"
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc reader.c -o reader
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./reader
Received Log: hi

```

5. IPC - Message Queues

1. Write a C program that creates a System V message queue and sends messages between two processes.

PROGRAM:

```
/*
Program to demonstrate message queue (Receiver)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : msgr.c
*/

// Header files
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

// Structure for message queue
struct msgbuf {
    long mtype;
    char mtext[100];
};

int main() {
    int msgid;
    struct msgbuf message;

    key_t key = ftok("msgfile", 65);

    msgid = msgget(key, 0666 | IPC_CREAT);

    msgrcv(msgid, &message, sizeof(message.mtext), 1, 0);

    printf("Received: %s\n", message.mtext);

    msgctl(msgid, IPC_RMID, NULL);

    return 0;
}
/*
Program to demonstrate message queue (Sender)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : msgsg.c
*/
```

```

// Include necessary header files
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

// Define the structure for the message buffer
struct msgbuf {
    long mtype;
    char mtext[100];
};

int main() {
    int msgid;
    struct msgbuf message;

    key_t key = ftok("msgfile", 65);

    msgid = msgget(key, 0666 | IPC_CREAT);

    message.mtype = 1;
    strcpy(message.mtext, "Hello from sender");

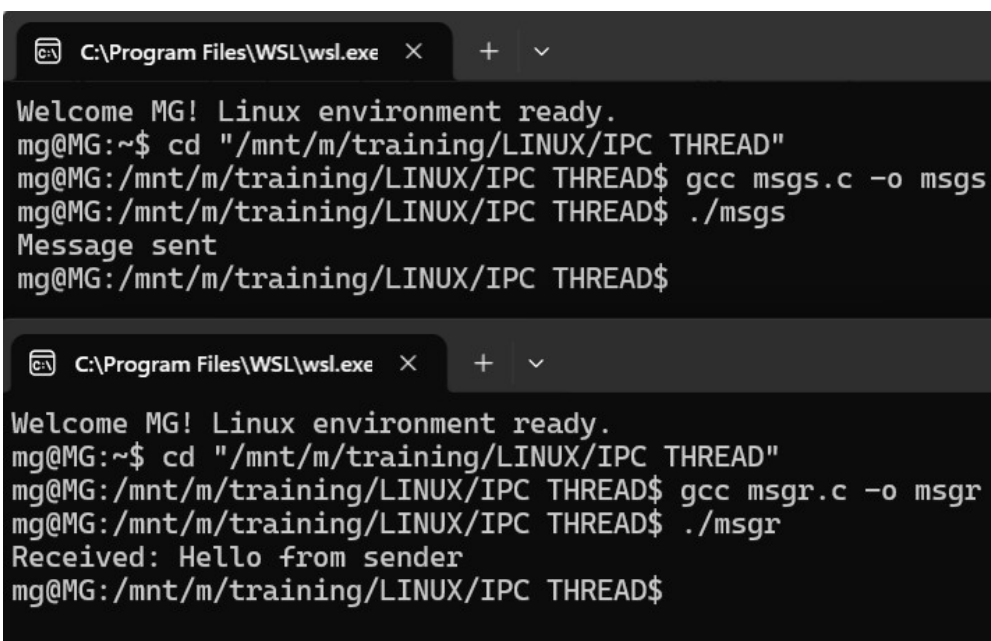
    msgsnd(msgid, &message, sizeof(message.mtext), 0);

    printf("Message sent\n");

    return 0;
}

```

OUTPUT:



```

C:\Program Files\WSL\wsl.exe X + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/IPC THREAD"
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc msgs.c -o msgs
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./msgs
Message sent
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

C:\Program Files\WSL\wsl.exe X + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/IPC THREAD"
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc msgr.c -o msgr
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./msgr
Received: Hello from sender
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

```

2. Implement a program where multiple clients send messages to a server process using a message queue, and the server processes them in order.

PROGRAM:

```
/*
Program for multiple clients to server (Server)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : mqs.c
*/

//Header files
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

//Message buffer structure
struct msgbuf {
    long mtype;
    char mtext[100];
};

int main() {
    int msgid;
    struct msgbuf message;

    //Generate unique key
    key_t key = ftok("msgfile", 75);

    //Create message queue and get its identifier
    msgid = msgget(key, 0666 | IPC_CREAT);

    while (1) {
        msgrcv(msgid, &message, sizeof(message.mtext), 1, 0);
        printf("Received from client: %s\n", message.mtext);
    }

    return 0;
}

/*
Program for multiple clients to server (Client)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : mqc.c
*/

//Header files
```



```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

//Message buffer structure
struct msgbuf {
    long mtype;
    char mtext[100];
};

int main() {
    int msgid;
    struct msgbuf message;

    key_t key = ftok("msgfile", 75);

    msgid = msgget(key, 0666 | IPC_CREAT);

    message.mtype = 1;

    printf("Enter message: ");
    fgets(message.mtext, sizeof(message.mtext), stdin);

    msgsnd(msgid, &message, sizeof(message.mtext), 0);

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc msg.c -o msg
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./msg
Message sent
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc mqs.c -o mqs
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./mqs
Received from client: hi

```

```

C:\Program Files\WSL\wsl.exe X + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/IPC THREAD"
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc msgr.c -o msgr
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./msgr
Received: Hello from sender
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc mqr.c -o mqr
cc1: fatal error: mqr.c: No such file or directory
compilation terminated.
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc mq.c -o mq
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./mq
Enter message: hi
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

```

3. Create a program that demonstrates message queue attributes, such as maximum message size and queue length.

PROGRAM:

```
/*
Program to display message queue attributes
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : mqa.c
*/

//Header files
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int main() {
    int msgid;
    struct msqid_ds buf;

    //Generate unique key
    key_t key = ftok("msgfile", 90);

    msgid = msgget(key, 0666 | IPC_CREAT);

    msgctl(msgid, IPC_STAT, &buf);

    printf("Maximum bytes allowed: %ld\n", buf.msg_qbytes);
    printf("Current messages in queue: %ld\n", buf.msg_qnum);
    printf("Last send PID: %d\n", buf.msg_lspid);
    printf("Last receive PID: %d\n", buf.msg_lrpid);

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc mqa.c -o mqa
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./mqa
Maximum bytes allowed: 16384
Current messages in queue: 0
Last send PID: 682
Last receive PID: 681
mg@MG:/mnt/m/training/LINUX/IPC THREAD$
```

6. Shared Memory

1. Write a C program that creates a shared memory segment and allows two processes to exchange data through it.

PROGRAM:

```
/*
Program to demonstrate shared memory (Reader)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : shmr.c
*/

//Header files
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    int shmid;
    char *str;

    //Generate unique key
    key_t key = ftok("shmfile", 65);

    //Create shared memory segment
    shmid = shmget(key, 1024, 0666 | IPC_CREAT);

    str = (char*) shmat(shmid, NULL, 0);

    printf("Data read from memory: %s\n", str);

    shmdt(str);

    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}
/*
Program to demonstrate shared memory (Writer)
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : shmw.c
*/

//Header files
#include <stdio.h>
```

```

#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    int shmid;
    char *str;

    key_t key = ftok("shmfile", 65);

    //Create shared memory segment
    shmid = shmget(key, 1024, 0666 | IPC_CREAT);

    str = (char*) shmat(shmid, NULL, 0);

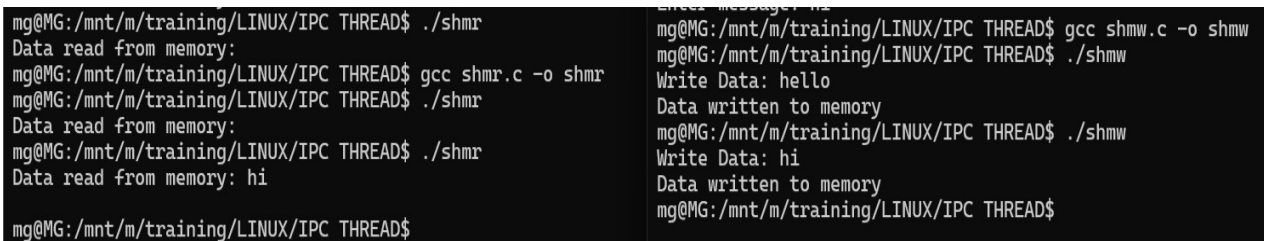
    printf("Write Data: ");
    fgets(str, 100, stdin);

    printf("Data written to memory\n");

    return 0;
}

```

OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./shmr
Data read from memory:
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc shmr.c -o shmr
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./shmr
Data read from memory:
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./shmr
Data read from memory: hi
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc shmw.c -o shmw
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./shmw
Write Data: hello
Data written to memory
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./shmw
Write Data: hi
Data written to memory
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

```

2. Implement a producer-consumer problem using shared memory for the buffer and semaphores for synchronization.

PROGRAM:

```

/*
Program for producer using shared memory + semaphore
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : pro.c
*/

#include <stdio.h>
#include <sys/ipc.h>

```

```

#include <sys/shm.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int shmid;
    int *buffer;

    key_t key = ftok("shmfile", 75);
    shmid = shmget(key, sizeof(int), 0666 | IPC_CREAT);

    buffer = (int*) shmat(shmid, NULL, 0);

    sem_t *sem = sem_open("sem", O_CREAT, 0666, 1);

    for (int i = 1; i <= 5; i++) {
        sem_wait(sem);

        *buffer = i;
        printf("Produced: %d\n", i);

        sleep(1);

        sem_post(sem);
    }

    return 0;
}
/*
Program for consumer using shared memory + semaphore
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : con.c
*/

//Header files
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int shmid;
    int *buffer;

    key_t key = ftok("shmfile", 75);

```

```

shmid = shmget(key, sizeof(int), 0666 | IPC_CREAT);

buffer = (int*) shmat(shmid, NULL, 0);

sem_t *sem = sem_open("sem", 0);

for (int i = 1; i <= 5; i++) {
    sem_wait(sem);

    printf("Consumed: %d\n", *buffer);

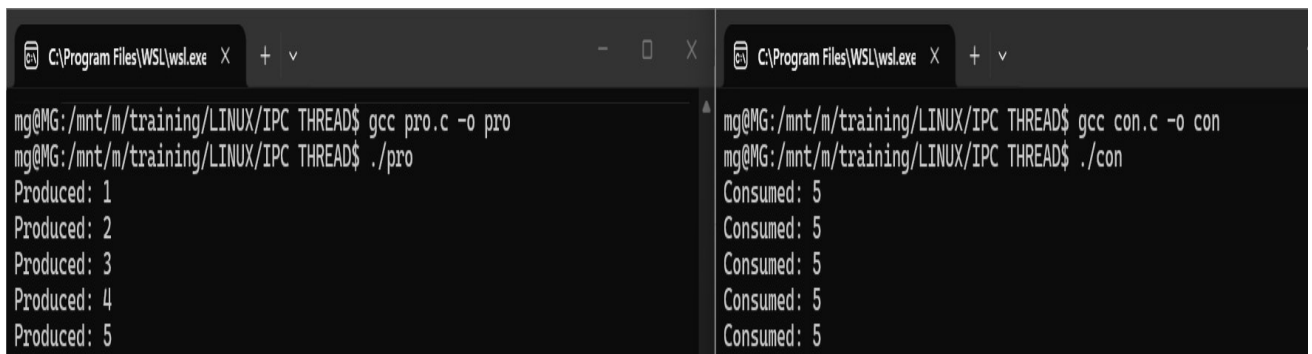
    sleep(1);

    sem_post(sem);
}

return 0;
}

```

OUTPUT:



The image shows two terminal windows side-by-side. The left window shows the output of the producer program (pro.c), which prints 'Produced: 1' through 'Produced: 5'. The right window shows the output of the consumer program (con.c), which prints 'Consumed: 5' five times. Both windows are running in a WSL environment with the prompt 'mg@MG:/mnt/m/training/LINUX/IPC THREAD\$'.

3. Create a program that demonstrates how to attach, detach, and remove a shared memory segment.

PROGRAM:

```

/*
Program to demonstrate attach, detach, remove shared
memory
Author    : MUTHUGANESH S
Date      : 13/03/2026
Filename  : shm.c
*/

//Header files
#include <stdio.h>

```

```

#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    int shmid;
    char *ptr;

    key_t key = ftok("shmfile", 90);

    // Create shared memory segment
    shmid = shmget(key, 1024, 0666 | IPC_CREAT);

    ptr = (char*) shmat(shmid, NULL, 0);

    printf("Attached shared memory\n");

    sprintf(ptr, "Test Message");

    printf("Data: %s\n", ptr);

    shmdt(ptr);
    printf("Detached shared memory\n");

    shmctl(shmid, IPC_RMID, NULL);
    printf("Removed shared memory\n");

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc shm.c -o shm
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./shm
Attached shared memory
Fatal glibc error: Xprintf_buffer_flush.c:70 (__printf_buffer_flush): assert
ion failed: buf->write_ptr != buf->write_end
Aborted (core dumped)
mg@MG:/mnt/m/training/LINUX/IPC THREAD$

```

7. Semaphores

1. Write a C program that uses POSIX semaphores to synchronize access to a shared resource among multiple processes.

PROGRAM:

```
/*
Program to synchronize shared resource using POSIX semaphore
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : sync.c
*/

//Header files
#include <stdio.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    sem_t *sem;

    // Create or open a named semaphore
    sem = sem_open("mysem", O_CREAT, 0666, 1);

    printf("Waiting to enter critical section...\n");
    sem_wait(sem);
    printf("Entered critical section\n");
    sleep(3);
    printf("Exiting critical section\n");
    sem_post(sem);
    sem_close(sem);
    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc sync.c -o sync
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./sync
Waiting to enter critical section...
Entered critical section
12Exiting critical section
mg@MG:/mnt/m/training/LINUX/IPC THREAD$
```


2. Implement a program where a semaphore controls access to a critical section, allowing only three processes at a time.

PROGRAM:

```
/*
Program to allow only 3 processes in critical section
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : lim.c
*/

//Header files
#include <stdio.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    //Create a named semaphore with an initial value of 3
    sem_t *sem;

    sem = sem_open("limitsem", O_CREAT, 0666, 3);

    printf("Process %d waiting...\n", getpid());

    sem_wait(sem);

    printf("Process %d entered\n", getpid());
    sleep(5);
    printf("Process %d leaving\n", getpid());

    sem_post(sem);

    sem_close(sem);

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc lim.c -o lim
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./lim
Process 879 waiting...
Process 879 entered
Process 879 leaving
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ |
```

3. Create a program that demonstrates the use of semaphore operations (`sem_wait`, `sem_post`) in a multi-threaded environment.

PROGRAM:

```
/*
Program to demonstrate sem_wait and sem_post with threads
Author   : MUTHUGANESH S
Date     : 13/03/2026
Filename : thr.c
*/

// Header files
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

// Semaphore declaration
sem_t sem;

// Worker thread function
void* worker(void* arg) {
    int id = *(int*)arg;

    sem_wait(&sem);

    printf("Thread %d in critical section\n", id);
    sleep(2);
    printf("Thread %d leaving\n", id);

    sem_post(&sem);

    return NULL;
}

int main() {
    pthread_t t1, t2, t3;
    int id1 = 1, id2 = 2, id3 = 3;

    sem_init(&sem, 0, 1);

    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);
    pthread_create(&t3, NULL, worker, &id3);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
}
```

```
    sem_destroy(&sem);  
  
    return 0;  
}
```

OUTPUT:

```
Process 075 leaving  
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ gcc thr.c -o thr  
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ ./thr  
Thread 1 in critical section  
Thread 1 leaving  
Thread 2 in critical section  
Thread 2 leaving  
Thread 3 in critical section  
Thread 3 leaving  
mg@MG:/mnt/m/training/LINUX/IPC THREAD$ |
```